

GLOSSARY

Glossary

Definitions for Claude Code terminology. Learn what agentic loop, compaction, CLAUDE.md, hooks, subagents, MCP, and other core concepts mean.

This glossary defines Claude Code terminology. Each entry links to the page where the concept is covered in depth. For model-level concepts like tokens, temperature, and RAG, see the [platform glossary](#).

A

Agent teams

Multiple independent Claude Code sessions coordinated by a team lead, with a shared task list and peer-to-peer messaging. Unlike [subagents](#), which run within a single session and report only to the parent, teammates each have their own context window and you can interact with any of them directly. Agent teams are experimental and must be enabled by setting

```
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Learn more: [Run agent teams](#)

Agentic coding

A workflow where the AI can read files, run commands, and make changes autonomously while you watch, redirect, or step away, as opposed to chat-based assistants that only respond with text you must apply yourself. Claude Code is agentic because it has [tools](#) that let it act, not just advise.

Learn more: [How Claude Code works](#)

Agentic harness

The tools, context management, and execution environment that turn a language model into a capable coding agent. Claude Code is the harness; Claude is the model inside it. The harness supplies file access, shell execution, permission gating, memory loading, and the loop that chains

actions together.

Learn more: [How Claude Code works](#)

Agentic loop

The cycle Claude works through for every task: gather context, take action, verify results, and repeat until done. Each tool use returns information that informs the next step. You can interrupt the loop at any point to redirect. Most extension points, including [hooks](#), [skills](#), and [MCP](#), plug into specific phases of this loop.

Learn more: [How Claude Code works](#)

Artifact

A live, interactive web page Claude Code publishes from your session to a private URL on claude.ai, so you can see output visually or share it inside your organization instead of reading terminal text. The page updates in place when the session republishes. Artifacts you create from Claude Code appear in the same gallery as artifacts created in claude.ai conversations, but their sharing stops at your organization and they cannot be made public.

Learn more: [Share session output as artifacts](#)

Auto memory

Notes Claude writes for itself based on your corrections and preferences, stored per git repository under `~/.claude/projects/`. All worktrees of the same repository share one auto memory directory. The first 200 lines or 25 KB of the `MEMORY.md` index loads at the start of every session. Auto memory is the Claude-written counterpart to [CLAUDE.md](#), which you write.

Learn more: [Auto memory](#)

Auto mode

A [permission mode](#) where a separate classifier model reviews actions in the background, so most run without approval prompts; explicit ask rules still prompt. The classifier blocks scope escalation, untrusted infrastructure, and [prompt injection](#). It never sees tool results, so injected instructions cannot influence its decisions. Auto mode is a research preview.

Learn more: [Eliminate prompts with auto mode](#)

B

Bare mode

A startup flag, `--bare`, that skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md. Only flags you pass explicitly take effect. Recommended for CI and scripted calls where you need identical behavior across machines regardless of local configuration.

Learn more: [Start faster with bare mode](#)

Bundled skills

Prompt-based playbooks included with Claude Code, such as `/batch`, `/code-review`, `/debug`, and `/loop`. Unlike built-in commands, which execute fixed logic, bundled skills give Claude a detailed prompt and let it orchestrate the work, so they can spawn agents, read files, and adapt to your codebase.

Learn more: [Bundled skills](#)

C

Channel

An **MCP server** that pushes events into your running session so Claude can react to things that happen while you're away from the terminal. Channels can be two-way: Claude reads an inbound event and replies back through the same channel. Telegram, Discord, and iMessage are included in the research preview.

Learn more: [Channels](#)

Checkpoint

A restore point created at each prompt you send. Claude Code snapshots files before every edit so a checkpoint can revert them. Press `Esc` twice or run `/rewind` to restore code, conversation, or both to an earlier point, or to summarize part of the conversation from a selected message.

Checkpoints are local to the session, separate from git, and don't track changes made through the Bash tool.

Learn more: [Checkpointing](#)

`.claude` directory

The directory where Claude Code reads project-scoped configuration: settings, hooks, skills, subagents, rules, and auto memory. A project has `.claude/` at its root; your user-level defaults are at `~/.claude/`.

Learn more: [The `.claude` directory](#)

CLAUDE.md

A markdown file of persistent instructions you write for Claude, loaded at the start of every session as a user message after the system prompt. Put project conventions, architecture notes, and “always do X” rules here. Project-root CLAUDE.md survives [compaction](#) and is re-read fresh from disk afterward.

You can place CLAUDE.md at project scope in `./CLAUDE.md` or `./.claude/CLAUDE.md`, at user scope in `~/.claude/CLAUDE.md`, or as [managed policy](#) for your organization. All discovered files are concatenated into context rather than overriding each other, ordered from broadest scope to most specific.

Learn more: [CLAUDE.md files](#)

Command

A reusable instruction you invoke by typing `/name` in the prompt. Built-in commands such as `/clear`, `/model`, and `/compact` control the session. You can define your own commands as files in `.claude/commands/`, or install them from a [plugin](#). [Skills](#) are the recommended way to package multi-step commands.

Learn more: [Commands](#) · [Skills](#)

Compaction

Automatic summarization of your conversation when the [context window](#) approaches its limit. Older tool outputs are cleared first, then the conversation is summarized. Project-root CLAUDE.md and auto memory survive compaction and reload from disk; instructions given only in conversation may be lost. Run `/compact` to trigger manually, optionally with a focus like `/compact focus on the API`

changes .

Learn more: [What survives compaction](#) · [When context fills up](#)

Context window

The working memory for a session, holding conversation history, file contents, command outputs, CLAUDE.md, auto memory, loaded skills, and system instructions. As you work, context fills up until **compaction** summarizes it. Run `/context` to see what's using space. For the underlying model concept, see the [platform glossary](#).

Learn more: [Explore the context window](#)

D

Dispatch

A phone-initiated task router that spawns a Claude Code session in the Desktop app when you send a coding task from the Claude mobile app. Your prompt routes to the right tool automatically.

Available on Pro and Max plans.

Learn more: [Sessions from Dispatch](#)

E

Effort level

A setting that controls how much of the adaptive-reasoning thinking budget Claude uses on each turn. Higher effort means more thinking tokens and deeper reasoning; lower effort is faster and cheaper. Effort is supported on Fable 5, on Opus 4.6 and later, and on Sonnet 4.6.

Learn more: [Adjust effort level](#)

Extended thinking

Visible step-by-step reasoning the model performs before responding. You can adjust it with the **effort level**, or cap thinking tokens with `MAX_THINKING_TOKENS` on models with a fixed thinking budget. Thinking appears in gray italic text in the terminal.

Learn more: [Use extended thinking](#)

H

Hook

A user-defined handler that executes automatically at a specific point in Claude Code's lifecycle, such as before a tool runs, after a file edit, or at session start. Handlers can be a shell command, HTTP endpoint, MCP tool, LLM prompt, or subagent. Hooks are deterministic: they fire at fixed lifecycle points rather than at the model's discretion.

A hook configuration has three levels:

- **Hook event:** the lifecycle point
- **Matcher:** filters which events fire it
- **Hook handler:** what runs

Learn more: [Get started with hooks](#) · [Hooks reference](#)

M

Managed settings

Settings enforced org-wide by IT or DevOps, delivered from Anthropic's servers through the admin console or deployed to devices at an OS-level path outside `~/.claude`. User and project settings cannot override managed settings. Server-managed delivery applies on [eligible configurations](#); see [Security considerations](#). Use this for security policies, compliance requirements, or standardized tooling across a fleet.

Learn more: [Server-managed settings](#) · [Settings files](#)

MCP (Model Context Protocol)

An open standard for connecting AI tools to external data sources and services. MCP servers give Claude new tools for Slack, Jira, databases, browsers, and hundreds of other integrations. You connect servers via `/mcp` or by adding them to `.mcp.json`. For the protocol itself, see the [platform glossary](#).

Learn more: [Model Context Protocol](#)

MCP Tool Search

A context-saving mechanism that defers MCP tool schemas until needed. Only tool names load at startup; Claude fetches the full schema on demand when it decides to use a specific tool. This keeps idle MCP servers from consuming much context.

Learn more: [Scale with MCP Tool Search](#)

N

Non-interactive mode

A mode that executes a single prompt and exits without a conversational session, invoked with `-p` or `--print`. Used for CI, scripts, and piping. The [Agent SDK](#) is the Python and TypeScript equivalent. Formerly called headless mode.

Learn more: [Run Claude Code programmatically](#)

O

Output style

A configuration that modifies Claude's system prompt to change response behavior, tone, or format. Output styles turn off the software-engineering-specific parts of the default system prompt, unlike [CLAUDE.md](#) which is delivered as a user message following the system prompt. Built-in styles include Default, Proactive, Explanatory, and Learning.

Learn more: [Output styles](#)

P

Permission mode

The baseline approval behavior for the session. Cycle with `Shift+Tab` in the CLI or use the mode selector in VS Code, Desktop, and claude.ai. Available modes are `default`, `acceptEdits`, `plan`, `auto`, `dontAsk`, and `bypassPermissions`.

Learn more: [Choose a permission mode](#)

Permission rule

A settings entry that allows, asks about, or denies a tool invocation based on the tool name and argument pattern. Rules are evaluated deny→ask→allow, first match wins. Permission rules are fine-grained controls layered on top of the broader [permission mode](#).

Learn more: [Configure permissions](#)

Plan mode

A [permission mode](#) where Claude researches and proposes changes without editing your source files. It can read, search, and run exploration commands, then presents a plan for approval before touching anything. Enter plan mode with `/plan` or by pressing `Shift+Tab`.

Learn more: [Analyze before you edit with plan mode](#)

Plugin

A bundle of skills, hooks, subagents, and MCP servers packaged as a single installable unit. Plugin skills are namespaced as `plugin-name:skill-name` so multiple plugins coexist. Distribute plugins across teams via a [marketplace](#).

Learn more: [Plugins](#)

Project trust

A dialog accepting a directory before Claude Code loads its configuration. Acceptance is saved per project directory, except your home directory, where trust is held for the current session only and the prompt reappears on each launch. Trust gates auto-installation of marketplace plugins and execution of project-defined hooks. Trusting a directory means its `.claude/settings.json`, `.mcp.json`, and other config files take effect.

Learn more: [The .claude directory](#)

Prompt injection

Hostile instructions embedded in a file, web page, or tool result that attempt to redirect Claude toward actions you never asked for. Claude Code's defenses include the permission system, command injection detection, and trust verification. [Auto mode](#) adds a server-side probe that scans

tool results for suspicious content and a classifier that never sees tool results, so injected text cannot influence its approval decisions.

Learn more: [Protect against prompt injection](#)

R

Remote Control

A way to continue a local Claude Code session from your phone or browser via claude.ai. Your code stays on your machine; only the UI is remote. Different from Claude Code on the web, which runs in a cloud sandbox.

Learn more: [Remote Control](#)

Rules

Modular instruction files in `.claude/rules/` that load alongside CLAUDE.md. A rule can be path-scoped with YAML `paths:` frontmatter so it only loads when Claude reads a matching file, keeping context lean until it's relevant.

Learn more: [Organize rules with .claude/rules/](#)

S

Sandboxing

OS-level filesystem and network isolation for the Bash tool. Commands run inside a boundary you define upfront, so Claude can work freely within it without per-command approval prompts. Sandboxing is a separate layer from [permission rules](#).

Learn more: [Sandboxing](#)

Session

A conversation tied to your current directory, with its own independent [context window](#). Sessions can be resumed with `claude -c`, forked with `--fork-session` to preserve history under a new session ID, or run in parallel across terminals. Running `/clear` starts a new session; the previous one stays stored and is available via `/resume`. Each session's transcript is stored under

`~/.claude/projects/` .

Learn more: [Work with sessions](#)

Settings layers

The hierarchy Claude Code reads configuration from, in precedence order from highest to lowest: **managed policy**, command-line arguments, local settings at `.claude/settings.local.json` , project settings at `.claude/settings.json` , then user settings at `~/.claude/settings.json` . Arrays merge across layers; scalars at a higher layer override lower ones.

Learn more: [Settings files](#)

Skill

A `SKILL.md` file containing instructions, knowledge, or a workflow that Claude adds to its toolkit. Claude loads a skill automatically when relevant, or you invoke it directly with `/skill-name` . Skills follow the Agent Skills open standard; Claude Code extends it with invocation control and subagent execution.

Skills are the recommended successor to custom commands. A file at `.claude/commands/deploy.md` and one at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way; existing command files continue to work.

Learn more: [Extend Claude with skills](#)

Subagent

A specialized AI assistant that runs in its own context window with a custom system prompt, specific tool access, and independent permissions. It works on a delegated task and returns a summary to the main conversation. Use subagents to keep large explorations out of your primary context or to run parallel research. Different from **agent teams**, where each agent is a full independent session you can talk to directly.

Built-in subagents include Explore, Plan, and general-purpose.

Learn more: [Create custom subagents](#)

Surface

Any place you access Claude Code: the CLI, VS Code, JetBrains, Desktop, or claude.ai. All surfaces share the same engine, so your CLAUDE.md, settings, and skills work the same way across them. Slack and the Chrome extension are integrations that connect to a surface rather than surfaces themselves.

Learn more: [Platforms and integrations](#)

T

Teleport

A command, `/teleport`, that pulls a cloud Claude Code session into your local terminal. Claude fetches the branch, loads the conversation history, and resumes from the web session's last state. The reverse direction is `--remote`, which sends a local task to run on the web.

Learn more: [From web to terminal](#)

Tool

An action Claude can take: read a file, edit code, run a shell command, search the web, spawn a subagent. Tools are what make Claude Code agentic. Without them, Claude can only respond with text. Each tool use returns a result that informs Claude's next decision in the [agentic loop](#).

Learn more: [Tools available to Claude](#)

Turn

One complete response from Claude within a [session](#). A turn begins when you send a message and ends when Claude finishes responding, with any number of [tool](#) calls in between. [Stop hooks](#) fire at the end of each turn. A session consists of many turns, and the [agentic loop](#) describes what happens inside one.

Learn more: [How Claude Code works](#)

V

Verification loop

How a session knows the work is actually done rather than just plausible. You give Claude a check it can run, such as a test suite, a build, or a screenshot comparison, and Claude iterates until the check passes instead of stopping after one attempt. A verification loop is the prerequisite for `/goal` , unattended runs, and **dynamic workflows**: without one, the only thing deciding the agent is finished is the agent itself.

Learn more: [Give Claude a way to verify its work](#)

W

Worktree isolation

An isolation mode that runs Claude in a separate git worktree under `.claude/worktrees/` , enabled with the `-w` flag or `isolation: worktree` in subagent config. Changes stay on a separate branch in a separate directory, so parallel agents don't overwrite each other's files.

Learn more: [Run parallel sessions with git worktrees](#)

Deprecated and renamed terms

These terms appear in older docs, blog posts, and community content. Use the current name when searching this site.

Old term	Now called	Notes
Headless mode	<u>Non-interactive mode</u>	Same <code>-p</code> flag, same behavior
Custom commands	<u>Skills</u>	<code>.claude/commands/</code> files still work
Slash commands	Commands	"Slash" dropped from product copy